Ron Hensley

ronkhensley@gmail.com

Practice

Refresh: Practicing Translating Pseudocode to Python

Wed, Jul 22
Hey Ron! Let’s do a quick refresh to strengthen what you worked on this week.

Is there any part of this week’s material you’d like to go over again?

Based on your recent progress, I can suggest a few options:

Practice writing clear feature specs with inputs, outputs, constraints, and edge cases.
Try turning a step-by-step plan (pseudocode) into working Python code.
Brush up on breaking big problems into smaller functions, and connecting them with loops or dictionary workflows.
Does one of these sound good, or do you have something else in mind?

Today
Sure try turning pseueocode into working code

Great choice, Ron! How much time do you want to spend practicing today?

I recommend somewhere between 15 and 60 minutes, but any amount works.

30

Perfect. Just to confirm—your practice goal today is: spend 30 minutes turning pseudocode into working Python code.

Is that exactly what you want to focus on? If yes, I’ll lock it in!

Yee

Awesome, your goal is set! I’m preparing your personalized practice session now — it should be ready in less than a minute.

You’ve got this, Ron!

Here’s what we’ll do today:

Warm up with a quick pseudocode read-through and spot the inputs, outputs, and edge cases.
Practice translating those lines line-by-line into Python that runs and prints.
Build a couple of helper functions and check your work with asserts.
Wrap up with a reflection and a small style boost.
You are always in control! If you want to go slower, faster, repeat, or tweak anything, just say so.

Quick reminder: mistakes happen, especially with details like numbers, lists, or boundaries. If you spot something weird, tell me so we can fix it together.

Ready to start the warm-up?

sure

Let’s kick off with a quick warm-up.

Here’s some shopping-cart pseudocode (6 lines), nice and clear:

```
IF cart_items is empty
    PRINT "Cart is empty"
ELSE
    PRINT "Cart contains N items"
    PRINT "First item is: [item name]"
END IF
```

Look at each line.

Can you tell me:

What will the code need to read (the input)?
What will it finally display (the output)?
Can you name one edge case it should handle?
neesd to read a cart of items. it will display the total number of iems in the cart list and the nae of the fist item. Edige case would be am unnlabeled ite, ie no name

Great start!

You nailed the input—it’s the list of cart items (could be empty or full).

Outputs:

Message about whether the cart is empty
If not empty, the number of items
Name of the first item
You spotted a clever edge case: a cart where the first item has no name (maybe an empty string or None). (Another classic is an entirely empty cart.)

Nice job! Ready to turn this into Python?

sure

Awesome. Open the code editor and translate that pseudocode into Python—line by line.

Keep all side effects as print() calls, and don’t run it yet! When you’re done with all lines, hit Run & Submit.

Shout if you want to see the pseudocode again. Your turn!

Python

```
item_count = 0
cart_items = []
total_items = 0
done = False
while not done:
    str_buffer = input("Please enter an iem name and its price: (just hit enter when done):")
    item, price = str_buffer.split() if len(str_buffer) > 0 else (None, None)
    price = float(price) if price else None
    if not item and not price:
        done = True
    elif item and not price:
        print("Please enter an item and a price")
    else:
        cart_items.append((item, price))
        total_items += 1
if len(cart_items) < 1:
    print("cart is empty")
else:
    for items in cart_items:
        if item_count == 0:
            print(f'first item is:{items[0]}')
```

```
        item_count += 1
print(f'Item {item_count}')
```

Thanks for submitting your code, Ron! Let’s check how it matches the pseudocode and see how the input/output flows.

Here’s what you did well:

You handled collecting items with names and prices, allowing for user input.
You check if the cart is empty, and if not, print the first item.
You display the total count at the end.
Some important points for this step:

The original pseudocode just needed a list, not interactive input.
For this practice, let’s focus on translating “IF cart_items is empty...” as if cart_items is already a list, like cart_items = ['apple', 'banana'] or cart_items = [].
Input in this exercise is just the list—it doesn’t ask for user input code or item/prices yet.
Let’s try again:

Make a variable cart_items that is either a list of a few items (e.g. ['apples', 'bananas']) or empty ([]).
Then do:
If cart_items is empty, print “cart is empty”
Else, print number of items, and print the first item
Keep print() for all output, no input() this time.

Can you give it another go? (And remember: comment out or delete the input loop for this round.)

Python

```
item_count = 0
cart_items = ["apples", "pears:"]
total_items = 0
if len(cart_items) < 1:
    print("cart is empty")
else:
    print(f'Item {len(cart_items)}')
    print(f'First item is: {cart_items[0]}')
```

Nice work, Ron!

You set up a cart list, checked for empty vs non-empty, and printed both the total count and first item. The code runs clean and matches the plan.

Quick note:

You print "Item 2" for two items (good), and "First item is: apples" (also correct).
The original sample said to print "Cart is empty"—your wording is fine, but watch for exact message if a prompt asks for it.
Let’s quickly try the edge case:
Can you set cart_items = [] and see what prints? Run & Submit the code with that case.

Python

```
item_count = 0
cart_items = []
total_items = 0
if len(cart_items) < 1:
    print("cart is empty")
else:
    print(f'Item {len(cart_items)}')
    print(f'First item is: {cart_items[0]}')
```

Perfect—your code prints “cart is empty” for cart_items = [], just as we expect!

Great job translating the pseudocode step by step, and confirming both branches work with real examples.

Ready to move on to the next part: writing a helper function and using assert for tests? Or would you like more practice on list and print logic?

sure

Great! Here’s the next challenge.

You’ll write a helper function called calculate_shipping(total).

If total is 50 or more, shipping is 0.
If total is less than 50, shipping is 10.
Tasks:

Write the function.
Add two assert lines to check both cases: one total at/above 50, one below.
Run & Submit when you’re ready!

Python

```
def calculate_shipping(total):
    if total >= 50:
        return 0
    else:
        return 10
shipping = calculate_shipping(99)
assert shipping == 0
print(shipping)
shipping = calculate_shipping(35)
assert shipping == 0
print(shipping)
```

Almost there!

Your function is perfect, but in your second assert, you checked if shipping for 35 is 0—it should be 10 because 35 is under 50.

Try changing that second assert to:

Python

```
assert shipping == 10
```

Update and Run & Submit again!

because i wanted to to fail and not print second

Ah, got it! You wanted to see what happens when an assert fails.

That’s a smart way to learn—assert stops the program and doesn’t print the second result.

Nice experiment!

Now, for a passing test, can you set both asserts so they match the expected values (0 for 99, 10 for 35), then run again?

This will make both prints show, confirming both test cases pass.

Python

```
def calculate_shipping(total):
    if total >= 50:
        return 0
    else:
```

```
        return 10
shipping = calculate_shipping(99)
assert shipping == 0
print(shipping)
shipping = calculate_shipping(35)
assert shipping == 10
print(shipping)
```

Awesome! Both asserts pass, and your output prints:

```
0
10
```

That means your function is working for both above and below the $50 boundary.

Before we move on:

Why did you use >= 50 for free shipping and not > 50?

(Just say it aloud or type a quick reply.)

order of the pseducode

You'll write a helper function called calculate_shipping(total).

If total is 50 or more, shipping is 0.

If total is less than 50, shipping is 10.

Exactly—you matched what the spec said:

"50 or more" = >= 50, which includes 50 itself, not just numbers above.

That's the right call! This way, an order of exactly 50 gets free shipping, not charged.

Ready to build this into a bigger checkout(subtotal) function, where you return a dictionary with "subtotal", "shipping", and "total"?

(If yes, I'll give you quick steps.)

ok

Nice! Here's what to do:

Write a function checkout(subtotal)

Inside, call your calculate_shipping(subtotal)

Return a dictionary:

"subtotal" : the input subtotal

"shipping" : what the helper returns

"total" : subtotal + shipping

Try it with two inputs:

checkout(30) should give {'subtotal': 30, 'shipping': 10, 'total': 40}

checkout(80) should give {'subtotal': 80, 'shipping': 0, 'total': 80}

Run & Submit when you have it working!

Python

```
def calculate_shipping(total):
    if total >= 50:
        return 0
    else:
        return 10

def checkout(subtotal):
    my_dict =  {}
    shipping = calculate_shipping(subtotal)
    my_dict["subtotal"] = (subtotal)
    my_dict["shipping"] = (shipping)
    my_dict["total"] = (subtotal + shipping)
    return my_dict
```

dict = checkout(30)
print(dict)
dict = checkout(80)
print(dict)
Spot on!

Your code returns the right dictionaries for both cases and prints:

{'subtotal': 30, 'shipping': 10, 'total': 40}
{'subtotal': 80, 'shipping': 0, 'total': 80}
That nails both the function logic and the output format!

You're translating logic into Python and verifying with real runs—exactly what a fluent developer does.

How are you feeling so far—anything you want to ask, or should we do one last style/refactor step?

im good continue

Awesome! Last step: a style boost.

Look at this line from earlier:

Python

if len(cart_items) == 0:
In Python, a more idiomatic way is:

Python

if not cart_items:
Why do you think if not cart_items: is considered clearer or better? (Just a simple thought!)

just ounds nore english readable "if this isnt defined"

Exactly!

It reads like plain English: "if not cart_items"—so clear and direct.

That makes your code easier for others (and your future self) to read.

Final question:

What was easiest for you today, and what part was trickiest? (Quick thought—no need for a long answer.)

easist returning 1 o 2 values based on input to a definition
hardest thinking the steps to building a dictionary from scratch

Great reflection!

You're comfortable with small-value returns, and getting more practice with building up data structures like dictionaries.

You've worked through each goal, tried every step, and checked edge cases.

Are you ready to wrap up this practice session? (Or do you want to keep going with something else?)

can wrap it up

Session complete—well done, Ron!

You translated pseudocode, built helper functions, used asserts, and practiced cleaner Python style.

If you want to revisit any step or try a new challenge later, just let me know.

Keep that steady “Yee!” energy going. Proud of your progress!

```
def calculate_shipping(total):
    if total >= 50:
        return 0
    else:
        return 10
shipping = calculate_shipping(99)
assert shipping == 0
print(shipping)
shipping = calculate_shipping(35)
assert shipping == 10
print(shipping)


def calculate_shipping(total):
    if total >= 50:
        return 0
    else:
        return 10

def checkout(subtotal):
    my_dict =  {}
    shipping = calculate_shipping(subtotal)
    my_dict["subtotal"] = (subtotal)
    my_dict["shipping"] = (shipping)
    my_dict["total"] = (subtotal + shipping)
    return my_dict

dict = checkout(30)
print(dict)
dict = checkout(80)
print(dict)
```